ISWE205P Computer Networks Lab

Lab Indicative Experiment 1

ONLINE DICTIONARY APPLICATION USING TCP (JAVA)

1. Aim

To write a TCP-based client–server program in Java where the client sends a word to the server and the server returns the meaning of the word stored in the dictionary.

2. Objective

- To understand TCP socket programming.
- To implement a real-time client–server dictionary lookup.
- To exchange data between client and server using Java's Socket and ServerSocket.

3. Theory

TCP (Transmission Control Protocol)

- Connection-oriented protocol
- Reliable byte-stream communication
- Ensures ordered and error-free delivery

Java Networking Classes

- ServerSocket – waits for client connection
- Socket – establishes connection between client and server
- BufferedReader, PrintWriter – used for input/output communication

4. Algorithm

Server

1. Create a ServerSocket on port 5000.
2. Accept client connection using accept().
3. Load dictionary words and meanings into a HashMap.
4. Receive word from client.
5. Search the word in HashMap.
6. Send back the meaning or an error message.
7. Close connection.

Client

1. Create socket connection to server on port 5000.
2. Accept a word from the user.
3. Send it to the server.
4. Receive the meaning from the server.
5. Display the meaning.
6. Close the socket.

Server Program code:

```
import java.io.*;
import java.net.*;
import java.util.HashMap;
public class DictionaryServer {
    public static void main(String[] args) {
        try {
```

```java
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Dictionary Server is running on port 5000...");
Socket socket = serverSocket.accept();
System.out.println("Client connected...");
BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

// Dictionary Data
HashMap<String, String> dictionary = new HashMap<>();
dictionary.put("computer", "An electronic device for storing and processing
data");
dictionary.put("network", "A group of interconnected computers");
dictionary.put("protocol", "A set of communication rules");
dictionary.put("java", "A high-level, object-oriented programming language");
dictionary.put("database", "A structured collection of data");
String word;

while ((word = in.readLine()) != null) {
    System.out.println("Word received from client: " + word);
    word = word.toLowerCase();
    if (dictionary.containsKey(word)) {
        out.println("Meaning: " + dictionary.get(word));
    } else {
        out.println("Word not found in dictionary!");
    }
}
```

```
          socket.close();
          serverSocket.close();
        }
          catch (Exception e)
          {
            e.printStackTrace();
          }
    }
}
```

Client Program Code

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class DictionaryClient {

    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            System.out.println("Connected to Dictionary Server...");

            BufferedReader in = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
```

```
        Scanner sc = new Scanner(System.in);
        String word;

        while (true) {
            System.out.print("\nEnter a word to search (or type exit): ");
            word = sc.nextLine();

            if (word.equalsIgnoreCase("exit")) {
                System.out.println("Closing connection...");
                break;
            }
            out.println(word);
            String meaning = in.readLine();
            System.out.println("Server Response: " + meaning);
        }
        socket.close();
    }
        catch (Exception e)
        {
         e.printStackTrace();
         }
  }
}
```

7. **Compile both programs**
   javac DictionaryServer.java
   javac DictionaryClient.java

8. **Run both programs**

   java DictionaryServer.java
   java DictionaryClient.java

9. **Sample Exceution – Server Side**

   Dictionary Server is running on port 5000...
   Client connected...
   Word received from client: computer
   Word received from client: protocol
   Word received from client: hello

10. **Sample Exceution – Client Side**

   Connected to Dictionary Server...

   Enter a word to search (or type exit): computer

   Server Response: Meaning: An electronic device for storing and processing data

   Enter a word to search (or type exit): protocol

   Server Response: Meaning: A set of communication rules

   Enter a word to search (or type exit): hello

   Server Response: Word not found in dictionary!

   Enter a word to search (or type exit): exit

   Closing connection...

****